



OOP using C++ Programming

Trainer : Priyanka R. Rangole

Email: priyanka.rangole@sunbeaminfo.com



Storage Classes

- Following are the storage classes in C/C++:
 1. auto
 2. static
 3. extern
 4. Register
- Storage class decides scope of variable and function and lifetime of the variable.

Scope in C++

- Scope decides the region/area/boundary where we can access variables/function.
- Following are the scope in C++:
 1. Block scope
 2. Function scope
 3. Prototype scope
 4. Class Scope
 5. Namespace Scope
 6. File Scope
 7. Program Scope

LifeTime in C++

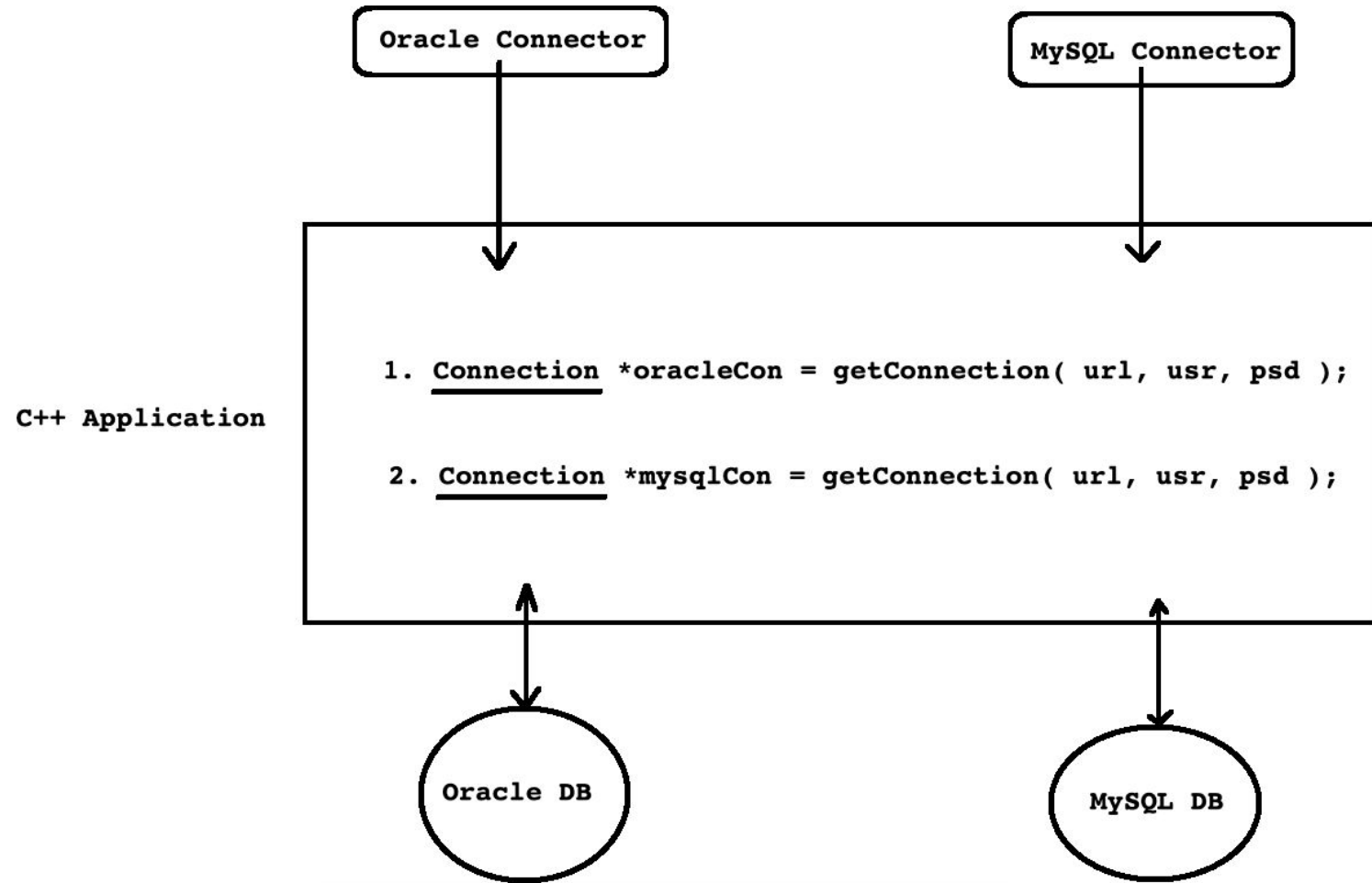
- Lifetime represents the existence of a variable/object inside RAM.
- Following are the lifetimes in C++:
 1. Automatic lifetime
 2. Static lifetime
 3. Dynamic lifetime

Scope resolution Operator ::

- Inside the same scope, we can not give the same name to the multiple variables. But name of local variable and global variables can be the same.
- If the name of local variable and global variable is same then preference is always given to the local variable.
- In C++, using the scope resolution operator, we can access the value of the global variable inside function.

```
int num1 = 10;  
int main( void ){  
    int num1 = 20;  
    printf("Num1 : %d\n", ::num1); //10
```

Need Of Namespace



Namespace

Namespace is C++ language feature that we can use:

1. To avoid name clashing/collision/ambiguity.
 2. To group functionally related / equivalent types together
- Using namespace keyword, we can define namespace. Consider the following code.

```
namespace na{  
    int num1 = 10;  
}  
int main( void ){  
    printf("Num1 : %d\n", na::num1);  
    return 0;  
}
```

- To access members of namespace, we should use :: operator

Namespace

- If the names of the namespaces are different then the names of members of namespace may/may not be the same.
- Consider following code.

```
namespace na{
    int num1 = 10;
    int num3 = 30;
}
namespace nb{
    int num2 = 20;
    int num3 = 40;
}
int main( void ){
    printf("Num1 : %d\n", na::num1); //10
    printf("Num3 : %d\n", na::num3); //30

    printf("Num2 : %d\n", nb::num2); //20
    printf("Num3 : %d\n", nb::num3); //40
    return 0;
}
```

Namespace

- If the names of the namespaces are the same then the names of members of namespace can not be the same. Consider the following code.

```
namespace na{
    int num1 = 10;  //OK
    int num3 = 30;  //OK
}
namespace na{
    int num2 = 20;  //OK
    int num3 = 40;  //Not OK
}
int main( void ){
    printf("Num1 : %d\n", na::num1); //10
    printf("Num3 : %d\n", na::num3); //30

    printf("Num2 : %d\n", na::num2); //20
    return 0;
}
```

Namespace

- We can define a namespace inside another namespace. It is called a nested namespace.

```
int num1 = 10;
namespace na{
    int num2 = 20;
    namespace nb{
        int num3 = 30;
    }
}
int main( void ){
    printf("Num1 : %d\n", ::num1); //10
    printf("Num2 : %d\n", na::num2); //20
    printf("Num3 : %d\n", na::nb::num3); //30
    return 0;
}
```

- Global members are considered as members of the global namespace.
- To access members of the global namespace, we can use only :: operator.

Using directive

- If we want to use members of namespace frequently then we should use “using directive”.

```
namespace na{
    int num1 = 10;
}
int main( void ){
    using namespace na;
    printf("Num1 : %d\n", num1); //10
    return 0;
}
```

```
namespace na{
    int num1 = 10;
}
int main( void ){
    int num1 = 20;
    using namespace na;
    printf("Num1 : %d\n", num1); //20
    printf("Num1 : %d\n", na::num1); //10
    return 0;
}
```

Using directive

- If we want to use members of namespace in multiple function then we should define using directive global.

```
namespace na{
    int num1 = 10;
}
using namespace na;
void show_record( void ){
    printf("Num1 : %d\n", num1);
}
void display_record( void ){
    printf("Num1 : %d\n", num1);
}
int main( void ){
    ::show_record( );
    ::display_record( );
    return 0;
}
```

Points to remember about Namespace

- We can give the same /different name to the namespaces.
- To access members of the namespace either we should use :: operator / using directive.
- We can not define namespace inside function/class. It should be global / it should be inside another namespace.
- We can not define the main function inside the namespace. It must be a member of the global namespace.
- We can not create object of namespace. It is used for grouping.
- std is a standard namespace in C++.
- Generally the names of the namespace should be in lower case.

Standard Stream of C++

- Stream is an abstraction(object) which is used to produce(write) and consume(read) data from source to destination.
- Console = Keyboard + Monitor.
- Following are the standard streams of C++ that is associated with console:
 1. cin : input stream represents keyboard
 2. cout : output stream represents monitor
 3. cerr : Unbuffered standard error stream
 4. clog : Buffered standard error stream

Character Output (cout)

- cout stands for character output.
- It is a standard output stream of C++ which represents a monitor. In other words, if we want to print the state of an object on the console/monitor then we should use cout.
- cout is the object of ostream class and ostream is the nickname/alias of `basic_ostream<char>` class.

```
typedef basic_ostream<char> ostream;
```

- cout is declared as extern object inside std namespace (hence `std::cout`) and std namespace is declared in `<iostream>` header file.
- cout is not a function hence to read data from the keyboard we must use insertion(`<<`) operator with it.

Character in (cin)

- cin stands for character input.
- It is the standard input stream of C++ which represents the keyboard. In other words, if we want to read data from the console/keyboard then we should use cin.
- cin is the object of istream class and istream is the nickname/alias of `basic_istream<char>` class.

```
typedef basic_istream<char> istream;
```

- cin is declared as extern object inside std namespace (hence `std::cin`) and std namespace is declared in `<iostream>` header file.
- cin is not a function hence to read data from the keyboard we must use the extraction(>>) operator with it.

Manipulator

- In C/C++, escape sequence is a character which is used to format the output.
- Example : '\n', '\t', '\b', '\a' etc.
- In C++, manipulator is a function which is used to format the output.
- Example : endl, setw, fixed, scientific, setprecision, dec, oct, hex etc.
- All the manipulators are declared in std namespace but header files are different
 1. To use endl include <iostream> header file
 2. To use remaining manipulators include <iomanip> header file
- Reference : <https://en.cppreference.com/w/cpp/io/manip>

Thank You

